

Least Recently Used Page Replacement Algorithm

1 Core Idea

Definition

The **Least Recently Used (LRU)** page replacement algorithm evicts the page that has not been used for the longest time.

LRU is based on locality. If a page was used recently, it is likely to be used again soon. If a page has not been used for a long time, it is a better victim.

LRU Rule

evict the resident page with the oldest last-use time

2 Why LRU Approximates Optimal

The optimal algorithm evicts the page whose next use is farthest in the future. LRU cannot know the future, so it uses the past as an estimate.

Observation	LRU assumption
Recently used pages often stay useful.	Keep recently used pages in memory.
Pages unused for a long time often remain unused.	Evict the page with the oldest last reference.

Main Intuition

LRU replaces impossible future knowledge with practical history: use the past to predict near-future behavior.

3 Linked-List Implementation

A direct LRU implementation keeps all pages in memory in a list.

front = most recently used, rear = least recently used

Event	List action
Page is referenced	Move that page to the front of the list.
Page fault occurs	Evict the page at the rear of the list.

Cost Problem

The list must be updated on every memory reference. Finding the page, removing it, and moving it to the front is too expensive for normal software implementation.

4 Worked Example

Problem

The LRU list is:

Front $[C, A, D, B]$ Rear

Page A is referenced. What happens to the list?

Solution

Page A was just used, so it becomes the most recently used page.

$$[C, A, D, B] \rightarrow [A, C, D, B]$$

Answer

Move A to the front. If a page fault happens next, LRU evicts B , the page at the rear.

5 Counter-Based Hardware Implementation

LRU can also be implemented with special hardware.

Counter Method

Use a hardware counter C that increments after each instruction. Each page table entry stores the value of C when that page is referenced.

The page table entry then contains a timestamp showing the page's most recent use.

Step	Action
After each instruction	Hardware increments counter C .
After each memory reference	Hardware stores current C in the referenced page's page table entry.
On page fault	OS scans the counters and finds the smallest value.
Victim	The page with the smallest counter is least recently used.

Counter Victim Rule

victim = page with minimum stored counter value

6 Counter Example

Problem

The current counter value is 900. The page table stores these last-use counters:

Page	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
Last-use counter	880	410	895	700

Which page is LRU?

Solution

LRU chooses the smallest last-use counter:

$$\min(880, 410, 895, 700) = 410$$

The value 410 belongs to page *B*.

Answer

Page *B* is least recently used, so LRU evicts *B*.

7 Why Exact LRU Is Expensive

Implementation	Cost
Linked list	Must update the list on every memory reference.
Counter method	Needs hardware counter and timestamp field in every page table entry.
Victim selection	OS may need to examine many page-table entries to find the smallest timestamp.

Practical Point

LRU is theoretically realizable, but exact LRU is costly. Many real systems use cheaper approximations instead.

8 Final Summary

One-Box Summary

LRU evicts the page unused for the longest time.
It approximates optimal by using recent past behavior.
Exact LRU needs costly per-reference tracking.
It can be implemented with a list or hardware counters, but both are expensive.